

VULNERABILITY ASSESSMENT AND PENETRATION TESTING REPORT

FOR

<http://testphp.vulnweb.com/>

FROM

SAUDA MOMIN

CONTENTS

Sr.no	Content	Page number
1.	Executive summary Summary Objective Report	3
	Tabular summary	4-5
3	Graphical Summary	6
4	Technical report	
4.1	Credentials stored in cookies	7-8
4.2	Sensitive data exposure via insecure file storage	9-11
4.3	Authentication bypass via sql injection	12-13
4.4	Transmission of sensitive information over insecure channel (http)	14-15
4.5	Credentials visible on sign up page	16-17
4.6	Sensitive information disclosure via directory browsing	18-19
4.7	Reflected cross site scripting in search bar	20-21
4.8	Error based sql injection	22-23
4.9	Missing functional access control	24-25
4.10	Information disclosure via phpinfo()	26-28
4.11	Missing X-Content type options header	29-30
4.12	Brute force attack on login functionality	31-32
4.13	Clickjacking	33-35
4.14	Information disclosure (X-powered by header)	36-37
4.15	Autocomplete Enabled on Login Form	38-39
5	Conclusion	40

SUMMARY

1.1 EXECUTIVE SUMMARY

INSTITUTE OF INFORMATION SECURITY (IIS) has assigned the task of vulnerability assessment and penetration testing (VAPT) on <http://testphp.vulnweb.com/>

This VAPT was performed on 3rd July 2025. The detailed report and our findings are described below.

1.2 OBJECTIVE

The objective of this test was to determine security vulnerabilities in the web server configuration and website running on the server. The tests were carried out assuming the identity of an attacker or with malicious intent. At the same time due care was taken not to harm the web server.

1.3 SCOPE

The scope of our work was limited to only <http://testphp.vulnweb.com/>. The vulnerabilities and our findings are described below.

TABULAR SUMMARY

The following tables summarize the vulnerability assessment of the server.

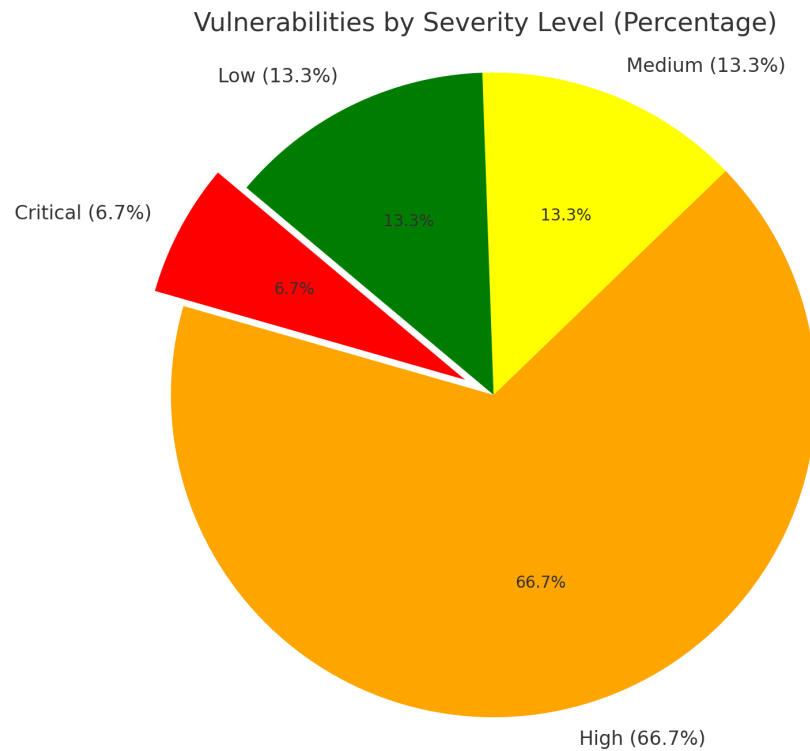
CATEGORY	DESCRIPTION
No of live host	1
No of vulnerabilities	15
No of critical vulnerabilities	1
No of high vulnerabilities	9
No of medium vulnerabilities	3
No of low vulnerabilities	2

Fig 2.1

SrNo.	Vulnerability Name	Severity	Ease of Exploitation
1	Credentials stored in cookies	Critical	Very Easy
2	Sensitive data exposure via insecure file storage	High	Very Easy
3	Authentication bypass via sql injection	High	Very Easy
4	Transmission of sensitive information over insecure channel (http)	High	Easy
5	Credentials visible on sign up page	High	Very Easy
6	Sensitive information disclosure via directory browsing	High	Very Easy
7	Reflected cross site scripting in search bar	High	Easy
8	Error based sql injection	High	Easy
9	Missing functional access control	High	Easy
10	Information disclosure via phpinfo	High	Very Easy
11	Missing X-Content-Type-Options Header	Medium	Easy
12	Brute force attack on login functionality	Medium	Easy
13	Clickjacking	Medium	Easy
14	Information disclosure (X-powered by header)	Low	Very Easy
15	Autocomplete Enabled on Login Form	Low	Easy

GRAPHICAL SUMMARY

The following pie chart graphically summaries the vulnerability assessment



3.3 – Pie chart showing the percentage distribution of vulnerabilities by severity

1. Credentials Stored in Cookies

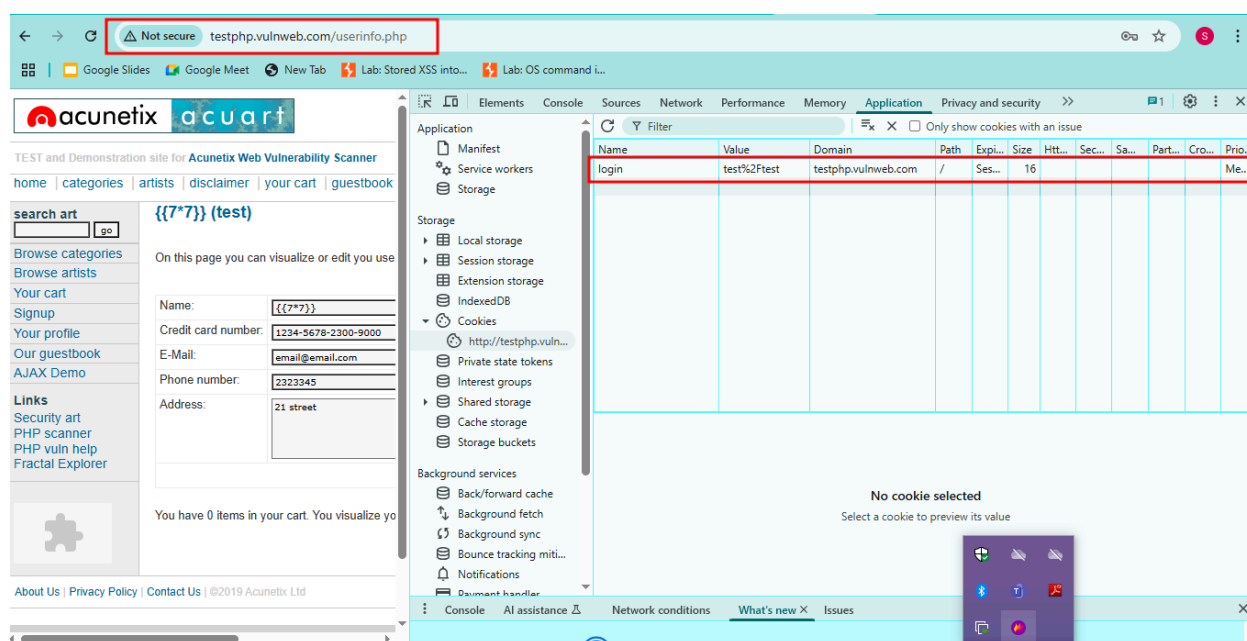
Affect on

Infected URL	http://testphp.vulnweb.com/userinfo.php
Infected Parameter	Cookies: login, password
Parameter type	Cookie
Attack vector	Browser inspection (no tools needed)

- Severity: Critical
- Ease of Exploitation: Very Easy (No special tools needed — anyone can view the cookie values through browser DevTools.)

Analysis

During testing of the site <http://testphp.vulnweb.com/>, I logged into the application and opened the browser's Developer Tools > Application > Cookies section. I observed that the application stores sensitive user credentials, such as the username and password, directly in the cookie values in plain text.



Impact:

- Exposes login credentials to anyone with access to the browser or device.
- Allows attackers to steal credentials via Cross-Site Scripting (XSS).
- Enables session hijacking if session or identity is derived from these cookie values.
- Credentials could be leaked in logs, referer headers, or sent in insecure requests if HTTPS is not enforced.
- Violates secure storage and transmission best practices for authentication data.

Root Cause:

- Insecure design choice to store sensitive data (credentials) in cookies.
- No server-side session token usage for authentication.
- Missing HttpOnly flag allows JavaScript access to cookies.
- Missing Secure flag allows cookies to be transmitted over unencrypted HTTP.
- No encryption or hashing of credentials in cookies.

Mitigation:

- Do not store usernames, passwords, or any credentials in cookies.
- Use server-generated session identifiers (tokens) instead of client-stored sensitive data.
- Set the HttpOnly flag on all cookies to prevent access from JavaScript.
- Use the Secure flag to ensure cookies are sent only over HTTPS.

2. Sensitive Data Exposure via Insecure File Storage

Affect on

Infected URL	http://testphp.vulnweb.com/pictures/credentials.txt
Infected Parameter	None (Direct file path access)
Parameter type	Static File Path (No user-controlled input)
Attack vector	Credentials.txt

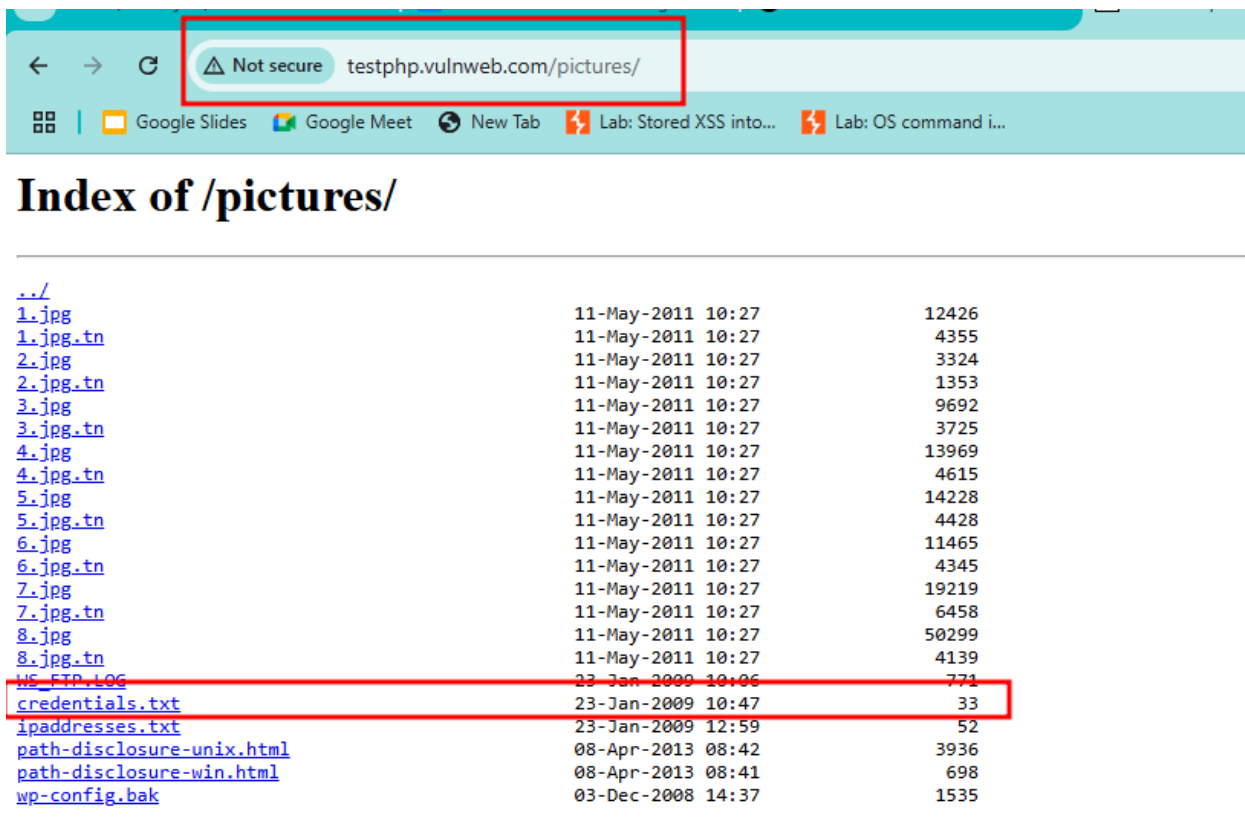
Severity: High

Ease of Exploitation:

Very Easy – Only requires knowledge of or brute-force discovery of the file path.

Analysis:

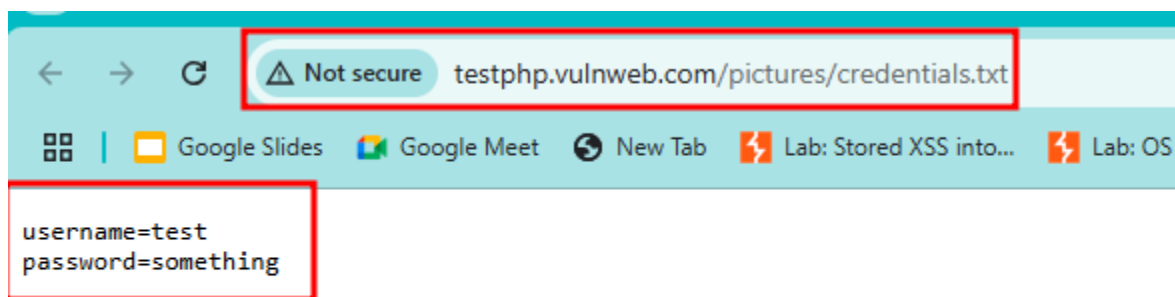
During testing of <http://testphp.vulnweb.com>, a file named `credentials.txt` was found in the `/pictures/` directory. The file was publicly accessible and opened without any login or restriction. It contained plain text credentials: a username and password. Access did not require any user input or parameter changes, confirming it's not an IDOR.



testphp.vulnweb.com/pictures/

Index of /pictures/

../		
1.jpg	11-May-2011 10:27	12426
1.jpg.tn	11-May-2011 10:27	4355
2.jpg	11-May-2011 10:27	3324
2.jpg.tn	11-May-2011 10:27	1353
3.jpg	11-May-2011 10:27	9692
3.jpg.tn	11-May-2011 10:27	3725
4.jpg	11-May-2011 10:27	13969
4.jpg.tn	11-May-2011 10:27	4615
5.jpg	11-May-2011 10:27	14228
5.jpg.tn	11-May-2011 10:27	4428
6.jpg	11-May-2011 10:27	11465
6.jpg.tn	11-May-2011 10:27	4345
7.jpg	11-May-2011 10:27	19219
7.jpg.tn	11-May-2011 10:27	6458
8.jpg	11-May-2011 10:27	50299
8.jpg.tn	11-May-2011 10:27	4139
HS_FTP.LOG	23-Jan-2009 10:06	771
credentials.txt	23-Jan-2009 10:47	33
ipaddresses.txt	23-Jan-2009 12:59	52
path-disclosure-unix.html	08-Apr-2013 08:42	3936
path-disclosure-win.html	08-Apr-2013 08:41	698
wp-config.bak	03-Dec-2008 14:37	1535



Impact:

- Leaks valid application credentials (username and password)
- Enables unauthorized access to application components
- Can lead to complete account or system compromise if credentials are reused
- Supports lateral movement if the attacker can log in elsewhere with the same credentials

Root Cause:

- Sensitive credentials are stored in a web-accessible directory (/pictures/)
- No access control or authentication is enforced on the file
- No file-level restrictions (e.g., .htaccess, permissions) are configured
- Application is using HTTP instead of HTTPS, increasing risk of interception

Mitigation:

- Remove sensitive files from publicly accessible directories
- Store credentials securely in server-side configuration files or environment variables
- Implement strict access control and file permission policies
- Disable directory browsing and apply deny rules for .txt and other sensitive file types
- Use HTTPS to protect against network-level sniffing

3. Authentication Bypass via SQL Injection

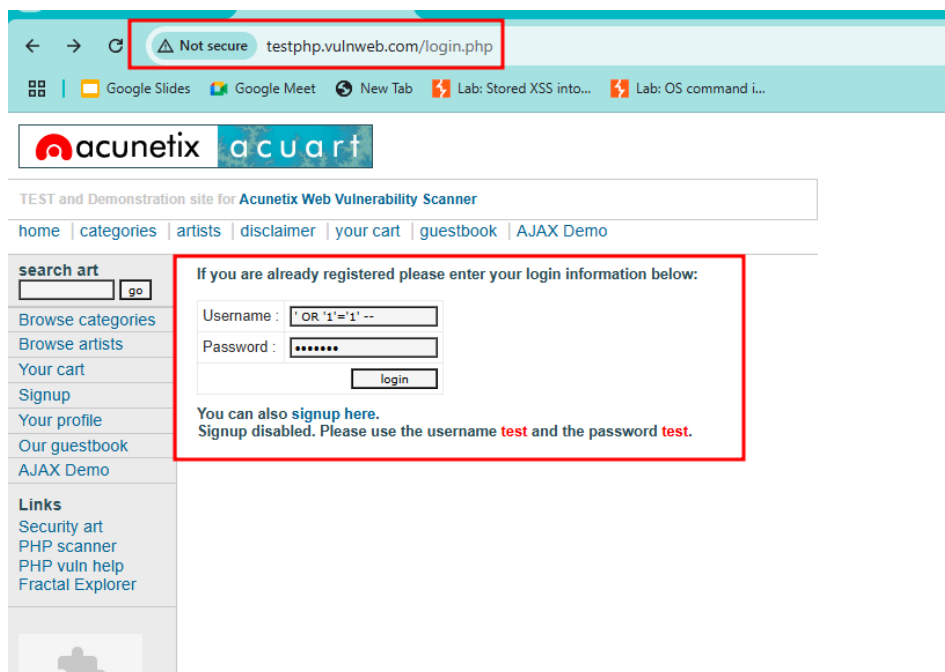
Affect on

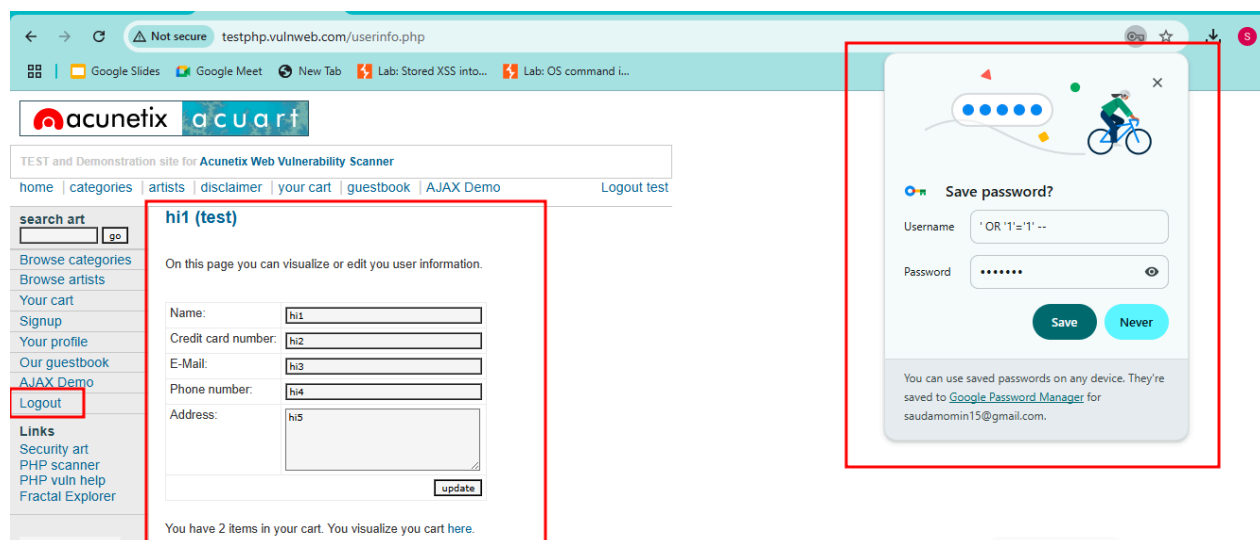
Infected URL	http://testphp.vulnweb.com/userinfo.php
Infected Parameter	username field in the login form
Parameter type	Post
Attack vector	' OR '1'='1' --

- **Severity:**High
- **Ease of Exploitation:** Very Easy – Requires only a simple payload in the login form, no special tools needed.

Analysis

Used the payload ' OR '1'='1' -- in the login form to bypass authentication. Access was granted to the user dashboard without knowing any real credentials. Confirmed that user input is not properly handled, resulting in SQL injection.





Impact:

- Allows attacker to log in as another user without knowing the password
- Leads to unauthorized access to personal user information

Root Cause:

- The SQL query directly uses user input without filtering or using prepared statements

Mitigation:

- Use **prepared statements** to safely handle user input in SQL queries

4. Transmission of Sensitive Information over Insecure Channel (HTTP)

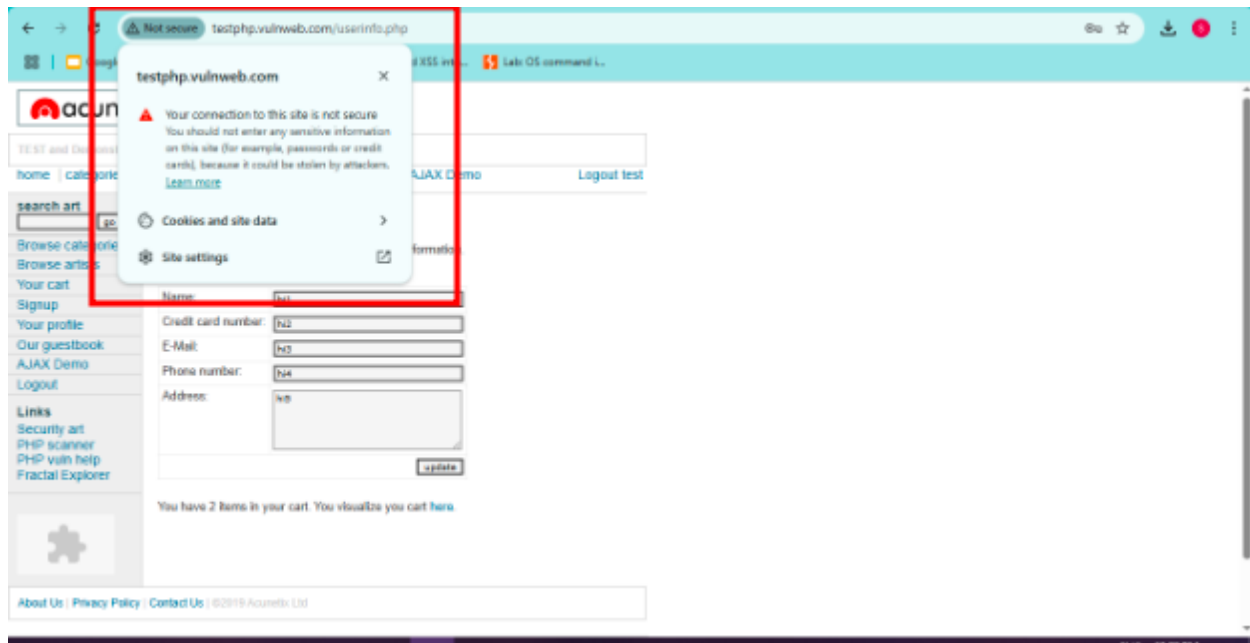
Affect on

Infected URL	http://testphp.vulnweb.com/userinfo.php
Infected Parameter	All user input field
Parameter type	User input fields (transmitted over insecure HTTP)
Attack vector	Plain HTTP

- **Severity:** High
- **Ease of Exploitation:** Easy – can be exploited by any attacker monitoring network traffic

Analysis:

Visited the user information page on <http://testphp.vulnweb.com/userinfo.php> and observed that the browser marked the site as “Not Secure.” Confirmed that the site uses HTTP instead of HTTPS while handling sensitive information such as credit card number, phone number, and email.



Impact:

- Sensitive user data (credit card, phone, email) can be intercepted
- Enables identity theft, financial fraud, or phishing
- Violates basic security practices and data protection requirements
- Damages user trust and application reputation

Root Cause:

- Application does not use HTTPS for secure data transmission
- No SSL/TLS encryption implemented on login or form submission

Mitigation:

- Implement HTTPS across the entire website using a valid SSL/TLS certificate
- Redirect all HTTP traffic to HTTPS
- Mark cookies as Secure and enable HSTS to enforce HTTPS

5. Credentials Visible on Signup Page

Affect on

Infected URL	http://testphp.vulnweb.com/signup.php
Infected Parameter	None (credentials are exposed directly in the page content or UI)
Parameter type	Static content (visible on page load)
Attack vector	Default credentials (username=test, password=test)

- **Severity:** High
- **Ease of Exploitation:** Very Easy – simply requires visiting the signup page to view credentials

Analysis

Visited the signup page at <http://testphp.vulnweb.com/signup.php> and observed that default credentials (username=test, password=test) were clearly shown. Used those credentials to log into the application and confirmed unauthorized access.

← → ↻ Not secure testphp.vulnweb.com/login.php

Google Slides Google Meet New Tab Lab: Stored XSS into... Lab: OS command i...

acunetix acuart

TEST and Demonstration site for Acunetix Web Vulnerability Scanner

[home](#) | [categories](#) | [artists](#) | [disclaimer](#) | [your cart](#) | [guestbook](#) | [AJAX Demo](#)

search art

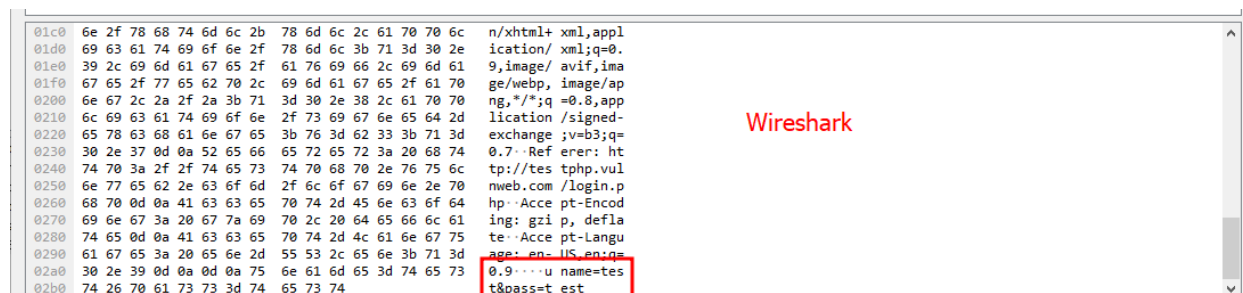
[Browse categories](#)
[Browse artists](#)
[Your cart](#)
[Signup](#)
[Your profile](#)
[Our guestbook](#)
[AJAX Demo](#)

Links
[Security art](#)
[PHP scanner](#)
[PHP vuln help](#)
[Fractal Explorer](#)

If you are already registered please enter your login information below:

Username :
 Password :

You can also sign up here.
 Signup disabled. Please use the username test and the password test.



The image shows a Wireshark packet capture of an HTTP GET request. The packet list on the left shows packet 0290 selected. The packet details pane on the right shows the 'Hypertext Transfer Protocol' section. The 'Request URI' field is highlighted with a red box, showing the path: `/test?name=test&pass=test`. The packet bytes pane at the bottom shows the raw data of the packet, with the same path highlighted in red.

```

01c0  6e 2f 78 68 74 6d 6c 2b 78 6d 6c 2c 61 70 70 6c  n/xhtml+xml,application/xhtml+xml;q=0.
01d0  69 63 61 74 69 6f 6e 2f 78 6d 6c 3b 71 3d 30 2e  9,image/avif,image/webp,image/apng,*/*;q=0.8,app
01e0  39 2c 69 6d 61 67 65 2f 61 76 69 66 2c 69 6d 61  ge/webp,image/apng,*/*;q=0.8,application/signed-ex
01f0  67 65 2f 77 65 62 70 2c 69 6d 61 67 65 2f 61 70  change;v=b3;q=0.7-Referer: http://test.php.vuln
0200  6e 67 2c 2a 2f 2a 3b 71 3d 30 2e 38 2c 61 70 70  nweb.com/login.php-Accept-Encoding: gzip, deflate
0210  6c 69 63 61 74 69 6f 6e 2f 73 69 67 6e 65 64 2d  70 74 2d 45 6e 63 6f 64 74 2d 4c 61 6e 67 75
0220  65 78 63 68 61 6e 67 65 3b 76 3d 62 33 3b 71 3d  te-Accept-Language: en-US,en;q=0.9...name=test
0230  30 2e 37 0d 0a 52 65 66 65 72 65 72 3a 20 68 74  age=en-US,en;q=0.9...name=test&pass=test
0240  74 70 3a 2f 2f 74 65 73 74 70 68 70 2e 76 75 6c  t&pass=test
0250  6e 77 65 62 2e 63 6f 6d 2f 6c 6f 67 69 6e 2e 70  70 74 2d 45 6e 63 6f 64 74 2d 4c 61 6e 67 75
0260  68 70 0d 0a 41 63 63 65 70 74 2d 45 6e 63 6f 64  70 74 2d 4c 61 6e 67 75 74 2d 4c 61 6e 67 75
0270  69 6e 67 3a 20 67 7a 69 70 2c 20 64 65 66 6c 61  70 74 2d 4c 61 6e 67 75 74 2d 4c 61 6e 67 75
0280  74 65 0d 0a 41 63 63 65 70 74 2d 4c 61 6e 67 75  70 74 2d 4c 61 6e 67 75 74 2d 4c 61 6e 67 75
0290  61 67 65 3a 20 65 6e 2d 55 53 2c 65 6e 3b 71 3d  70 74 2d 4c 61 6e 67 75 74 2d 4c 61 6e 67 75
02a0  30 2e 39 0d 0a 0d 0a 75 6e 61 6d 65 3d 74 65 73  70 74 2d 4c 61 6e 67 75 74 2d 4c 61 6e 67 75
02b0  74 26 70 61 73 73 3d 74 65 73 74

```

Impact:

- Allows unauthorized access to a default user account
- Leaks login details to everyone, bypassing account control
- May lead to further exploitation of user-specific features and stored data
- Violates secure design principles and user data isolation

Root Cause:

- Application displays hardcoded or default credentials on a public page
- No access control or validation on sensitive information display

Mitigation:

- Remove hardcoded credentials from public pages
- Do not display or expose login credentials in the UI or source code
- Enforce unique user registration and account separation
- Educate developers on secure coding and credential management

6. Sensitive Information Disclosure via Directory Browsing

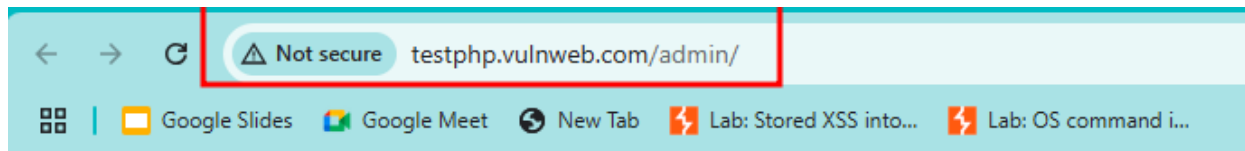
Affect on

Infected URL	http://testphp.vulnweb.com/admin/ http://testphp.vulnweb.com/CVS/
Infected Parameter	vulnerability exists due to open directory listing
Parameter type	Directory (static path)
Attack vector	/admin/, /CVS

- Severity:- High
- Ease of Exploitation:- Very Easy – anyone with the URL can access and view files

Analysis

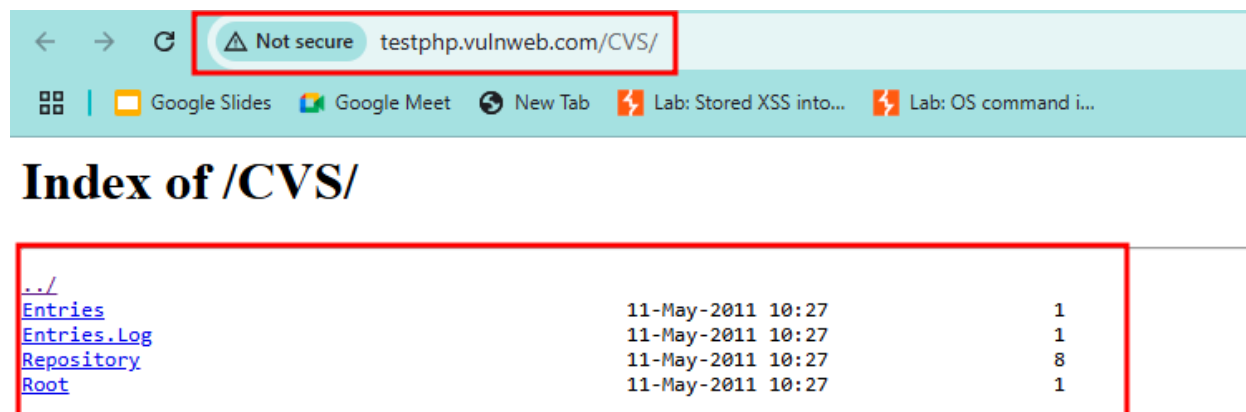
Accessed the URL <http://testphp.vulnweb.com/admin/> and found that directory listing was enabled. The file `create.sql` was listed openly and accessible without any restriction. This exposes sensitive backend information and poses a serious security risk.



Index of /admin/

../		
create.sql	11-May-2011 10:27	523

Accessed <http://testphp.vulnweb.com/CVS/> and found that CVS version control metadata files are listed and downloadable. These files can expose internal paths and source structure.



Impact:

- Disclosure of sensitive internal or administrative information
- Exposed files may include config data, internal docs, or credentials
- Can aid attackers in recon and planning further attacks
- Violates data protection and least privilege principles

Root Cause:

- Directory listing is enabled on the web server
- No access control or index file to block directory viewing

Mitigation:

- Disable directory browsing in the web server configuration
- Apply access controls to admin folders
- Move sensitive files outside the web root if not meant to be accessed directly
- Monitor public directories for unintended file exposure

7. Reflected Cross-site scripting in search bar

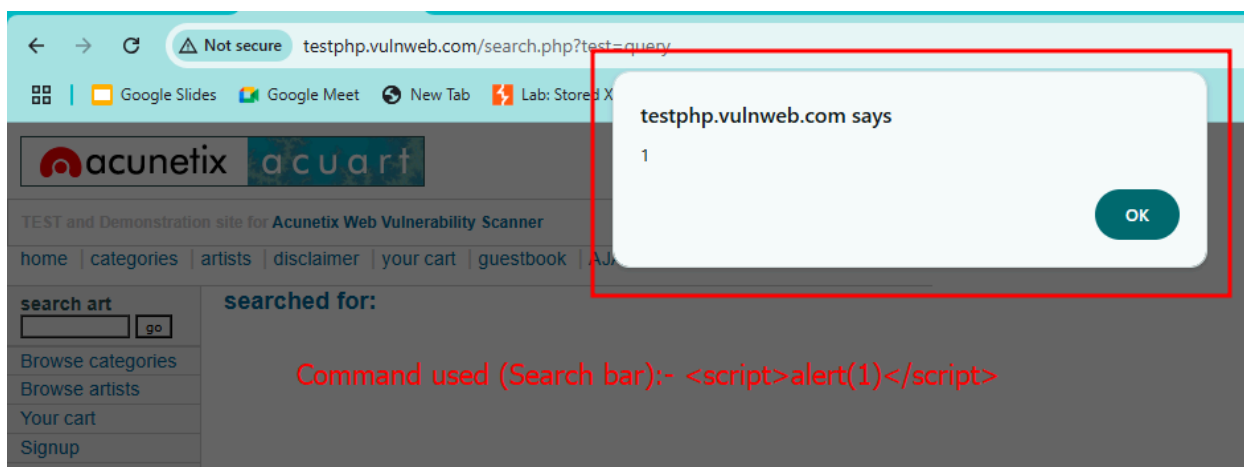
Affect on

Infected URL	http://testphp.vulnweb.com/search.php?test=query
Infected Parameter	Search input field
Parameter type	User input
Attack vector	<script>alert(1)</script>

- **Severity:** High
- **Ease of Exploitation:** Easy – attacker only needs access to the search form

Analysis

Entered <script>alert(1)</script> into the search bar on http://testphp.vulnweb.com/search.php. After submitting the form, the script executed in the browser, confirming a reflected XSS vulnerability. This was done through the input field, not the URL



Impact:

- Executes malicious scripts in users' browsers
- Can steal cookies, hijack sessions, deface pages, or redirect users
- May lead to further client-side attacks

Root Cause:

- User input from the form is directly inserted into the HTML output without encoding or filtering
- Application does not sanitize or validate search field input before displaying it

Mitigation:

- Encode all output that includes user input (e.g., HTML encode)
- Use proper input validation and context-aware output escaping
- Implement a strong Content Security Policy (CSP)

8. Error-Based SQL Injection

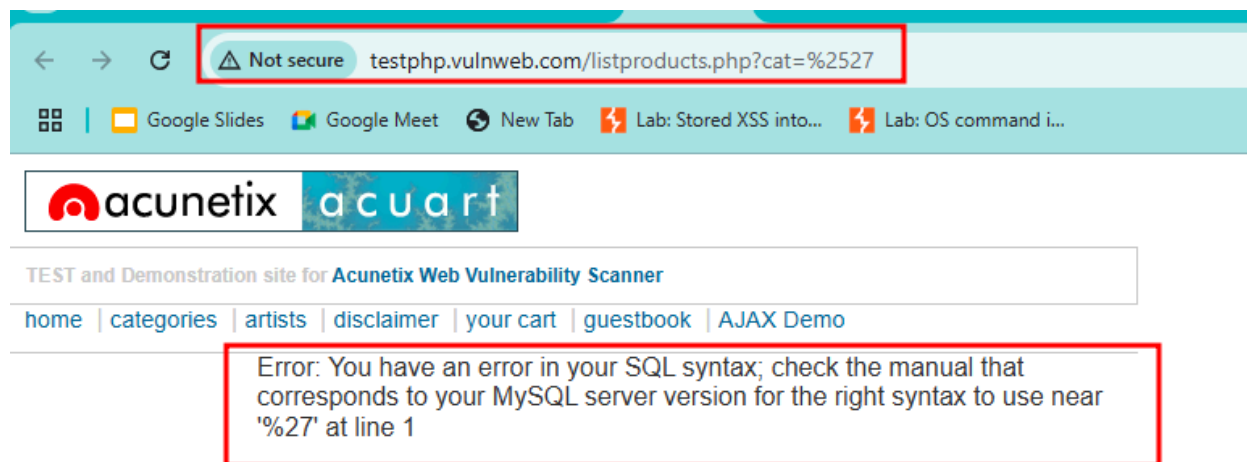
Affect on

Infected URL	http://testphp.vulnweb.com/listproducts.php?cat=%2527
Infected Parameter	Cat
Parameter type	Get
Attack vector	A double-encoded payload %2527 was injected into the cat parameter of the URL.

- **Severity:** High
- **Ease of Exploitation:** Easy – only requires altering the URL with encoded characters

Analysis

Accessed the URL `http://testphp.vulnweb.com/listproducts.php?cat=%2527` with a double-encoded single quote. The server returned a MySQL syntax error, confirming that user input is being injected into an SQL query without sanitization. This proves the application is vulnerable to error-based SQL injection.



Impact:

- Exposes backend SQL error messages
- Confirms SQL injection point
- Can be used to further extract or manipulate database data
- Increases the risk of full database compromise

Root Cause:

- Application accepts and directly executes unsanitized input
- SQL errors are returned in the HTTP response
- No input validation or parameterized queries used

Mitigation:

- Sanitize and validate all user inputs
- Use prepared statements with parameter binding
- Suppress detailed SQL errors in production
- Apply a WAF to detect and block SQLi patterns

9. Missing Functional Access Control

Affect on

Infected URL	http://testphp.vulnweb.com/userinfo.php?id=2
Infected Parameter	id
Parameter type	Get
Attack vector	A GET request to userinfo.php with a different id parameter (e.g., id=2

- Severity:- High
- **Ease of Exploitation:-** Easy

Analysis: -

I accessed the URL <http://testphp.vulnweb.com/userinfo.php?id=2> and was able to see sensitive details of a user without being logged in or authorized. There was no login or session verification required, and changing the id parameter allowed access to another user's data.

The presence of editable fields like name and credit card information indicates that an attacker could not only view but also attempt to update user data.

acunetix acuart

TEST and Demonstration site for Acunetix Web Vulnerability Scanner

home | categories | artists | disclaimer | your cart | guestbook | AJAX Demo | Logout test

search art go

Browse categories
Browse artists
Your cart
Signup
Your profile
Our guestbook
AJAX Demo
Logout

Links
Security art
PHP scanner
PHP vuln help
Fractal Explorer

..../WEB-INF/web.xml? (test)

On this page you can visualize or edit you user information.

Name:/WEB-INF/web.xml?
Credit card number: 1331
E-Mail: h@x0r
Phone number: 1234567890
Address: earth

update

You have 0 items in your cart. You visualize you cart [here](#).

Impact:

- Unauthorized access to personal and sensitive user data.
- Potential data leakage including name, email, credit card number, and address.
- Enables horizontal privilege escalation by changing the id.
- Can lead to identity theft, financial fraud, or full account compromise.

Root Cause:

- Absence of access control logic on the server-side.
- Application relies on client-side inputs (like id) without validating user session or role.
- No session-based ownership checks before displaying or allowing edit access to data.

Mitigation:

- Implement server-side access control checks for every sensitive operation.
- Always verify that the requesting user has permission to view or modify the requested resource.
- Use session-based user identification instead of accepting user IDs from the URL.

10. Information Disclosure via phpinfo()

Affect on

Infected URL	http://testphp.vulnweb.com/secured/phpinfo.php
Infected Parameter	None (direct access vulnerability)
Parameter type	None (no input required; exposed resource)
Attack vector	/secured/phpinfo.php

- **Severity:-** High
- **Very Easy :-** No authentication, user interaction, or special tools required. Simply visiting the URL discloses all information.

Analysis

I directly accessed the URL <http://testphp.vulnweb.com/secured/phpinfo.php> in the browser without logging in or providing any credentials. The page loaded successfully and displayed the complete phpinfo() output.



Directive	local value	master value
default_charset	no value	no value
default_mimetype	text/html	text/html
define_syslog_variables	Off	Off
disable_classes	no value	no value
disable_functions	no value	no value
display_errors	On	On
display_startup_errors	Off	Off
doc_root	no value	no value

This tells you that errors are shown directly in the browser , a misconfiguration.

error_prepend_string	no value	no value
error_reporting	2039	2039
expose_php	On	On
extension_dir	/usr/local/lib/php/20050022	/usr/local/lib/php/20050022
file_uploads	On	On
highlight.bg	#FFFFFF	#FFFFFF

This means the server includes PHP version info in response headers. It's a minor info leak but still important.

session.serialize_handler	php	php
session.use_cookies	On	On
session.use_only_cookies	Off	Off
session.use_trans_sid	0	0

That means **session IDs can be passed via URL**, which opens up **session fixation** attacks.

SERVER_SOFTWARE	Apache/2.2.3 (FreeBSD) DAV/2 PHP/5.1.6 mod_ssl/2.2.3 OpenSSL/0.9.7e-p1
SERVER_NAME	acuart
SERVER_ADDR	192.168.0.5
SERVER_PORT	80
REMOTE_ADDR	192.168.0.26
DOCUMENT_ROOT	/var/www/acuart/

This reveals **internal IP and hostname**, useful for attackers.

Impact:

- Discloses PHP version and all loaded modules, which can be used to identify known vulnerabilities.
- Reveals sensitive configuration settings such as `display_errors`, `session.use_only_cookies`, and `expose_php`.
- Exposes local file system paths, internal IP addresses, and environment variables.
- Assists attackers in planning targeted exploitation, privilege escalation, and further reconnaissance.

Root Cause:

- The `phpinfo()` page is publicly accessible in the production environment.
- No authentication or access restriction is implemented for sensitive diagnostic files.
- Default PHP settings (`display_errors`, `expose_php`) are not hardened.

Mitigation

- Remove or disable `phpinfo()` files from production servers.
- If needed for debugging, restrict access using IP-based rules or authentication.
- Apply least privilege and proper access controls to diagnostic and internal files.

11. Missing X-Content-Type-Options Header

Affect on

Infected URL	http://testphp.vulnweb.com/
Infected Parameter	N/A
Parameter type	HTTP Response Header
Attack vector	X-Content-Type-Options: nosniff header, the browser may incorrectly render or execute this content.

- **Severity:** Medium
- **Ease of Exploitation:** Easy - The attacker only needs the ability to upload a file or trigger a response from the server that could be misinterpreted.

Analysis

I ran a Nikto scan on <http://testphp.vulnweb.com> and found that the X-Content-Type-Options header is missing, which can allow browsers to perform MIME-type sniffing and potentially interpret files as executable scripts, increasing the risk of content-sniffing attacks like XSS.

```

nikto -h http://testphp.vulnweb.com/
- Nikto v2.5.0

+ Target IP:      44.228.249.3
+ Target Hostname: testphp.vulnweb.com
+ Target Port:    80
+ Start Time:    2025-07-07 22:52:52 (GMT-4)

+ Server: nginx/1.19.0
+ /: Retrieved x-powered-by header: PHP/5.6.40-38+ubuntu20.04.1+deb.sury.org+1.
+ /: The anti-clickjacking X-Frame-Options header is not present. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Frame-Options
+ /: The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a different fashion than the MIME type. See: https://www.netsparker.com/web-vulnerability-scanner/vulnerabilities/missing-content-type-header/

```

Impact

- Browsers may ignore the declared MIME type and interpret files based on their content.
- This can lead to the execution of malicious scripts.
- Increases the risk of client-side attacks, such as cross-site scripting (XSS).

Root Cause

- The server does not set the X-Content-Type-Options: nosniff header in its HTTP responses.
- This allows browsers to perform MIME-type sniffing and potentially execute untrusted content.

Mitigation

- Configure the server to include the following header in all HTTP responses:

X-Content-Type-Options: nosniff

12. Brute Force Attack on Login Functionality

Affect on

Infected URL	http://testphp.vulnweb.com/login.php
Infected Parameter	Password (in login form)
Parameter type	User input (text field)
Attack vector	Different passwords were submitted through Burp Suite Intruder. This behavior can be used to detect valid password or successful login attempts.

- **Severity:** Medium
- **Ease of Exploitation:** Easy – requires only basic payloads and Burp Suite

Analysis

The login page is vulnerable to brute force attack. It is possible to perform a password brute force on the login page.

Request	Payload	Status code	Response received	Error	Timeout	Length	Comment
26	a'or'=''	200	297			6292	
27	!=''	302	460			258	
28	a'or'=''	302	411			378	
29	a'or'=''	200	485			6292	
30	!=''	302	410			258	
31	!=''	302	483			258	
32	!=''	302	308			258	
33	password	302	319			258	
34	Pass	302	323			258	
35	test	200	478			6292	

Request: Response

```

1 POST /userinfo.php HTTP/1.1
2 Host: testphp.vulnweb.com
3 Content-Length: 20
4 Cache-Control: max-age=0
5 Upgrade-Insecure-Requests: 1
6 Origin: http://testphp.vulnweb.com
7 Content-Type: application/x-www-form-urlencoded
8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/121.0.6167.160 Safari/537.36
9 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
10 Referer: http://testphp.vulnweb.com/login.php
11 Accept-Encoding: gzip, deflate, br
12 Accept-Language: en-US,en;q=0.9
13 Connection: keep-alive
14
15 username=test&pass=test
  
```

Results Positions Payloads Resource pool Settings							
Filter: Hiding 30x, 40x and 50x responses							
Request	Payload	Status code	Response received	Error	Timeout	Length	Comment
17	a' or '1'='1	200	347			6375	
18	test	200	303			6359	
19	TEST	200	316			6351	
20	TeSt	200	312			6327	
21	TEsT	200	294			6324	
22	TEsT	200	311			6326	

Impact:

- Enables attackers to guess passwords and gain unauthorized access
- High risk of account compromise, especially if users use weak or reused passwords
- Can be automated easily, leading to large-scale attacks

Root Cause:

- No protection mechanisms against repeated failed login attempts
- Application does not detect or block brute force behavior
- Lack of account lockout, IP throttling, or CAPTCHA

Mitigation:

- Implement account lockout or delay after multiple failed login attempts
- Use CAPTCHA to block automated login attempts
- Monitor and alert on brute force patterns from the same IP or user
- Enforce strong password policies to reduce chances of successful guessing

13.Clickjacking

Affect on

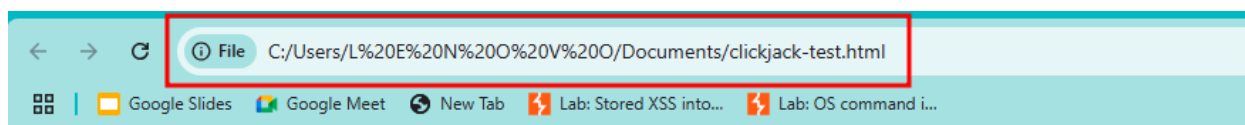
Infected URL	http://testphp.vulnweb.com
Infected Parameter	No specific parameter – entire page is vulnerable
Parameter type	N/A
Attack vector	The attacker creates a malicious HTML page that embeds the target site inside an invisible or partially transparent <iframe> and tricks the victim into clicking on UI elements.

- Severity: Medium
- Ease of Exploitation: Easy – No authentication or special access required

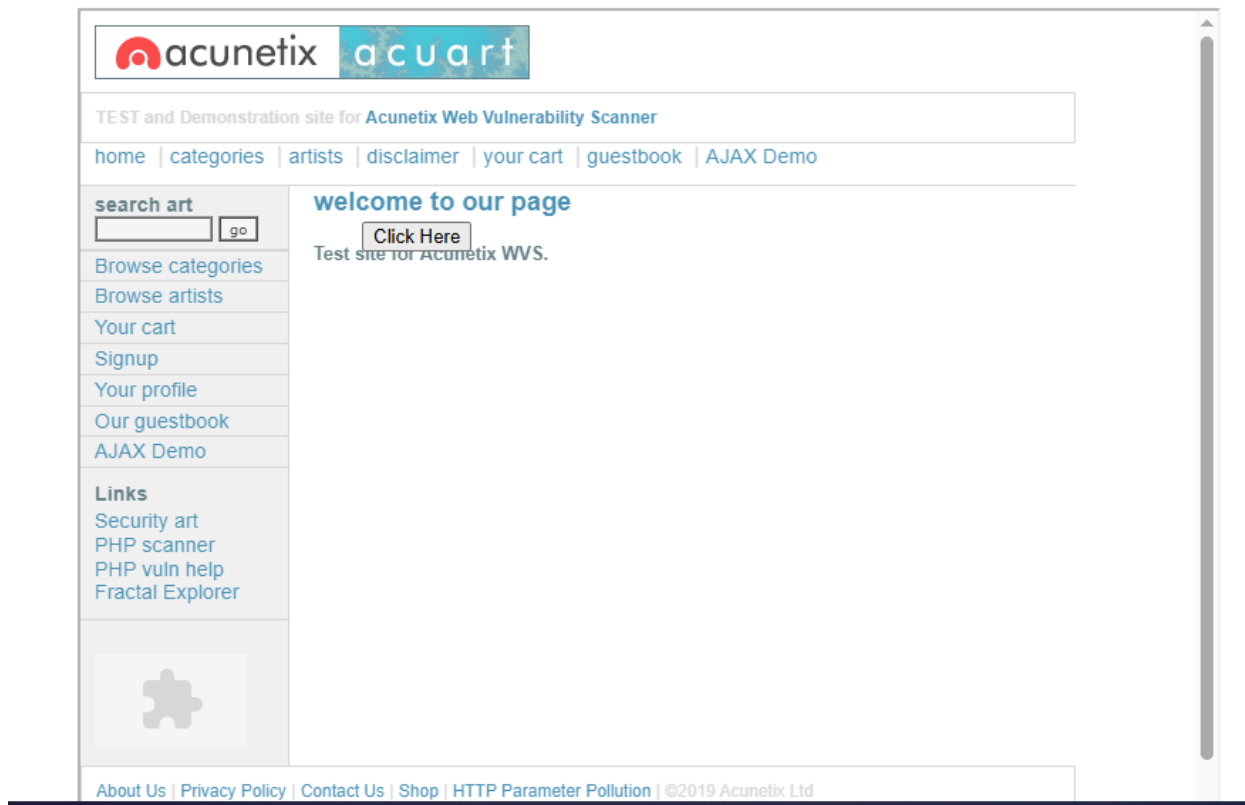
Analysis

Created an HTML file with an <iframe> pointing to http://testphp.vulnweb.com.
Open the HTML file in a browser. The website successfully loaded within the iframe, confirming there is no X-Frame-Options or CSP (Content-Security-Policy) header to prevent framing.

NAME	DATE MODIFIED	TYPE	SIZE
Machines	6/5/2025 9:40 AM	File folder	
clickjack-test	7/3/2025 6:38 PM	Chrome HTML Do...	1 KB
csrf-logout	6/28/2025 11:29 AM	Chrome HTML Do...	1 KB
kali	5/22/2025 9:13 PM	Open Virtualizatio...	79 KB
Page1	6/2/2025 12:17 PM	Chrome HTML Do...	1 KB



Click the button below to win ₹1000!



Impact

- A malicious site can load the target website in a hidden iframe and trick users into interacting with it unknowingly.
- Attackers can perform unauthorized actions on behalf of users by overlaying deceptive UI elements (e.g., fake buttons).
- Can lead to serious consequences like unintended clicks on "buy," "delete," or "submit" buttons if the site contains such actions.
- Erodes user trust and may lead to phishing-style attacks through visual deception.

Root Cause

- The server does not set the X-Frame-Options or Content-Security-Policy (CSP) header to control how and where its content can be embedded.
- As a result, any site can embed this application in an iframe.

Mitigation

- Add the HTTP response header X-Frame-Options: DENY to completely prevent embedding in iframes.
- Alternatively, use X-Frame-Options: SAMEORIGIN if framing is needed within the same origin only.

14. Information Disclosure (X-Powered-By Header)

Affect on

Infected URL	http://testphp.vulnweb.com/
Infected Parameter	X-Powered-By response header
Parameter type	HTTP Response Header
Attack vector	curl -I http://testphp.vulnweb.com/

- **Severity:** Low (but can lead to higher risk when chained with other vulnerabilities)
- **Ease of Exploitation:** Very Easy

Analysis

To identify this vulnerability, I used the `curl -I http://testphp.vulnweb.com/` command to fetch only the HTTP response headers from the target server. This quickly revealed the X-Powered-By header in the server's response, which disclosed that the application is running on PHP version 5.6.40.

Since this version is outdated and no longer supported, it can help attackers find and exploit known vulnerabilities specific to that version. No authentication or special tools were required—just a basic HTTP request. This confirmed that the server is leaking internal implementation details due to a misconfiguration, making it a clear case of information disclosure.

```
(root@kali)-[/home/kali]
# curl -I http://testphp.vulnweb.com/

HTTP/1.1 200 OK
Server: nginx/1.19.0
Date: Thu, 03 Jul 2025 10:51:07 GMT
Content-Type: text/html; charset=UTF-8
Connection: keep-alive
X-Powered-By: PHP/5.6.40-38+ubuntu20.04.1+deb.sury.org+1
```

Impact:

- Reveals backend technology (PHP) and exact version.
- Helps attackers search for and exploit known vulnerabilities.
- Increases risk of targeted attacks like RCE or LFI.
- Aids reconnaissance in multi-step exploitation.

Root Cause:

- `expose_php` is enabled in PHP configuration by default.
- Web server is not configured to suppress sensitive headers.
- Outdated PHP version (5.6.40) still in use.

Mitigation:

- Set `expose_php = Off` in `php.ini`.
- Suppress the header in web server config:
 - Nginx: `fastcgi_hide_header X-Powered-By;`
 - Apache: `Header unset X-Powered-By`
- Upgrade PHP to a supported version (e.g., PHP 8.x).
- Apply regular server hardening and security patches.

15. Autocomplete Enabled on Login Form

Affect on

Infected URL	http://testphp.vulnweb.com/login.php
Infected Parameter	uname, pass
Parameter type	Form fields (text, password)
Attack vector	Browser stores credentials → Autofill → Accessed by another user or stolen by XSS

- **Severity:-** Low
- **Ease of exploitation:-** Easy – No special tools or privileges needed; can be abused by browser or script

Analysis

I inspected the login form on <http://testphp.vulnweb.com/userinfo.php> and found that autocomplete was set to "on", which means the browser can store and autofill credentials - a security risk on shared devices or if XSS is present.

```

<!-- begin content -->
<!-- InstanceBeginEditable name="content_rgn" -->
</div id="content">
  <div class="story">
    <h3>If you are already registered please enter your login information below:</h3>
    <div>
      <form name="loginform" method="post" action="userinfo.php">
        </div>
      <div class="story">
    </div>
  </div>
<!-- InstanceEndEditable -->
<!--end content -->
<div id="navBar">

```

```

2: input
  acceptCharset: ""
  accessKey: ""
  action: "http://testphp.vulnweb.com/userinfo.php"
  attributeStyleMap: StylePropertyMap {size: 0}
  attributes: NamedNodeMap {0: name, 1: method, 2: action, name: name, method: me
  autocomplete: "on"
  autofocus: false
  baseURI: "http://testphp.vulnweb.com/login.php"
  childElementCount: 1
  childNodes: NodeList(3) [text, table, text]
  children: HTMLCollection [table]
  classList: DOMTokenList [value: '']
  className: ""
  clientHeight: 82
  clientLeft: 0

```

Impact

- Credentials may be autofilled by the browser.
- Risk of credential theft on shared or public systems.
- Stored credentials can be extracted if a cross-site scripting (XSS) vulnerability exists.

Root Cause

- autocomplete="on" is set or omitted from the form and input fields.
- Browser defaults to storing and auto filling sensitive input fields.

Mitigation

- Add autocomplete="off" to the <form> tag.
- Also apply autocomplete="off" to the username and password <input> fields individually.
- Review all authentication-related forms to ensure autocomplete is disabled where appropriate.

Conclusion

The testing of <http://testphp.vulnweb.com/> found many serious security problems that put the website and its users at risk. These include saving passwords in an unsafe way, SQL injection, cross-site scripting (XSS), missing access control, exposed sensitive data, and sending information over HTTP instead of HTTPS. There are also other issues like open folders, exposed system files, and missing cookie protections.

Most of these problems happen because the site does not check user input properly, has weak login/session controls, and the server is not secured well. If attackers find these issues, they could steal data, log in without permission, or take full control of the system.

To make the website safer, it's important to:

- Write secure code
- Use HTTPS for all pages
- Check and clean all user inputs
- Use strong login and session security
- Secure the server settings
- Follow security standards like OWASP Top 10 and CIS Benchmarks